Міністерство освіти і науки України
Київський національний університет імені Тараса Шевченка

КУРСОВИЙ ПРОЄКТ

з дисципліни «Теорія програмування»

SIPL Language Toolkit

Парсер, інтерпретатор, візуалізація та семантичні терми

Виконав:
студент групи ТТП-32
Кашинський Владислав

Київ - 2026

Зміст

1 Мета та постановка задачі 2

2 Архітектура 2

3 Формальна граматика SIPL 2

4 Типізоване AST та SourceSpan 3

5 Інтерпретатор і подієва модель 3

6 Семантичні терми 3

7 Візуалізація 4

8 Командний інтерфейс 4

9 Тестування та демонстрація 4

10 Відтворюваність та обмеження 5

11 Висновок 5

Використані матеріали 5

1. Мета та постановка задачі

Мета курсового проєкту - створити розширюваний набір інструментів для навчальної мови SIPL, у якому граматика, парсер, AST, виконання, візуалізація та семантика працюють через стабільні програмні інтерфейси. Реалізація не прив'язана до одного варіанта практичної роботи й підтримує цілий клас SIPL-програм.

Для виконання обрано модулі, наведені в таблиці 1.

Таблиця 1: Заявлені модулі проєкту

ID	Зміст	Бали
CORE-01	Машинозчитувана граматика, позитивні та негативні тести	0.5
CORE-02	Парсер і типізоване AST із позиціями у тексті	1.5
CORE-03	Інтерпретатор SIPL із лімітом кроків	2.0
CORE-04	Подієва модель виконання та trace	1.0
CORE-05	CLI та структурований JSON-режим	1.0
VIS-01	Parse tree у форматах Text, JSON і DOT	1.5
VIS-02	AST та порівняння Parse Tree $\leftrightarrow$ AST	1.0
SEM-01	Автоматична побудова семантичного терму	2.0
	Заявлена сума	10.5

За правилами курсового проєкту підсумкова оцінка обмежується 10 балами.

2. Архітектура

Інструментарій реалізовано мовою Python. Опис мови зберігається у формальній граматиці Lark. На її основі парсер будує конкретне синтаксичне дерево, яке потім перетворюється у власне типізоване AST. Саме AST передається інтерпретатору, модулю семантичних термів і засобам візуалізації.

Для роботи з усіма можливостями передбачено єдиний командний інтерфейс. Окремо реалізовано локальну вебсторінку, що використовує ті самі модулі й не дублює логіку синтаксичного аналізу або виконання програми.

Потік даних має вигляд

$$\text{Source} \longrightarrow \text{Parse Tree} \longrightarrow \text{AST} \longrightarrow \begin{cases} \text{Interpreter} \longrightarrow \text{State + Trace}, \\ \text{Semantic Builder} \longrightarrow \text{Semantic Term}, \\ \text{Visualizer} \longrightarrow \text{Text/JSON/DOT}. \end{cases}$$

Граматика визначена в одному файлі. Інтерпретатор, семантичний модуль та візуалізатори не аналізують початковий текст повторно, а використовують спільні AST-вузли. Це усуває дублювання логіки й дозволяє надалі під'єднати IDE або debugger.

3. Формальна граматика SIPL

Програма обмежується ключовими словами `begin` та `end`. Послідовність операторів розділяється крапкою з комою. Підтримано присвоювання, блок, умовний оператор, цикл і `skip`.

```
program: "begin" stmt_list? "end"
stmt_list: statement (";" statement)* ";"?

statement: assignment | if_stmt | while_stmt | block | "skip"
assignment: NAME ":=" expr
if_stmt: "if" bool_expr "then" statement "else" statement
while_stmt: "while" bool_expr "do" statement
```

Для арифметики реалізовано операції `+`, `-`, `*`, `div`, `mod`; для умов - порівняння, `not`, `and`, `or`. Граматика задає пріоритет і ліву асоціативність операторів. Однорядкові та блокові коментарі ігноруються лексером.

Набір тестів містить 11 коректних і 11 некоректних синтаксичних прикладів, що перевищує мінімальний критерій CORE-01.

4. Типізоване AST та SourceSpan

Парсер Lark спочатку будує конкретне дерево, після чого модуль `parser.py` створює власні незмінні AST-вузли. Класи `Number` і `Variable` представляють числа та змінні. Операціям відповідають `UnaryExpr`, `BinaryExpr`, `Comparison` і `BoolBinary`. Оператори подано класами `Assignment`, `Skip`, `If`, `While` та `Block`. Усе дерево об'єднує вузол `Program`.

Кожен вузол має `SourceSpan`: початковий і кінцевий рядок, колонку та зміщення у вихідному тексті. Завдяки цьому помилки й події можна точно пов'язати з фрагментом програми. AST серіалізується у стабільний JSON, незалежний від внутрішнього формату бібліотеки Lark.

5. Інтерпретатор і подієва модель

Стан SIPL-програми подано відображенням

$$State = Var \to Int,$$

реалізованим словником Python. Присвоювання оновлює компоненту стану, послідовність виконує оператори зліва направо, `if` обирає одну гілку, а `while` повторює тіло, доки умова істинна.

Читання неініціалізованої змінної й ділення на нуль породжують зрозумілу runtime-помилку з позицією у програмі. Ліміт кроків перериває потенційно нескінченний цикл.

Перед виконанням оператора створюється `BeforeStatement`. Після нього додається `AfterStatement`. Зміну значення фіксує `VariableChanged`, а перевірку умови в `if` або `while` позначає `ConditionEvaluated`. Виконання завершує подія `ProgramFinished`, що містить кінцевий стан.

Кожна подія має `SourceSpan`. Trace доступний як об'єкт Python, текст CLI та JSON, тому його можна повторно використати у debugger або візуалізаторі.

6. Семантичні терми

Семантичний терм будується рекурсивно з AST. Наприклад, для програми перевірки парності отримано:

$$WH(S^2(gr, N^{\Rightarrow}, 1), AS^N(S^2(sub, N^{\Rightarrow}, 2))) \cdot$$
$$IF(S^2(eq, N^{\Rightarrow}, 1), AS^R(1), AS^R(0)).$$

Тут S^2 - двомісна суперпозиція, $N^{\Rightarrow}$ - розіменування, AS^N - присвоювання, WH - цикл, IF - розгалуження, а $\cdot$ - послідовна композиція. Отже, позначення відповідають матеріалам курсу й попереднім практичним роботам.

7. Візуалізація

Parse tree та AST можна вивести як звичайне текстове дерево з відступами й позиціями вузлів. Для програмного опрацювання передбачено JSON, а для графічного подання - формат Graphviz DOT, з якого будуються SVG або PNG.

Окрема команда порівнює кількість вузлів. Parse tree зберігає правила граматики й токени, тоді як AST вилучає пунктуацію та проміжні синтаксичні категорії, залишаючи лише семантично значущі конструкції.

8. Командний інтерфейс

Реалізовано команди:

Команда	Призначення
`check`	перевірка синтаксичної коректності
`parse`	побудова типізованого AST
`run`	виконання та виведення кінцевого стану
`trace`	покроковий журнал виконання
`semantic`	побудова семантичного терму
`tree`	візуалізація parse tree
`ast`	візуалізація AST
`compare`	порівняння конкретного й абстрактного синтаксису

JSON-відповіді мають поле `schema_version = "1.0"`. Код завершення 0 означає успіх, 2 - синтаксичну або I/O-помилку, 3 - runtime-помилку.

Локальний вебінтерфейс запускається командою `sipl-web`.

Адреса сайту: `127.0.0.1:8000`. Він дає змогу ввести програму й стан, виконати аналіз, переглянути AST, семантичний терм і trace. Зовнішні сервіси не потрібні.

9. Тестування та демонстрація

Повний автоматичний набір містить 44 тести. Результат:

```
44 passed
TOTAL: 670 statements, 97 missed, coverage 84%
```

Покриття перевищує вимогу 80% для ядра. Перевірено синтаксис, пріоритети операцій, SourceSpan, виконання, коротке замикання умов, помилки, ліміт кроків, trace, CLI, JSON, DOT і семантичні терми.

Для демонстрації підготовлено три програми. Піднесення до степеня при $X = 2$ і $Y = 5$ дає $R = 32$. Алгоритм Евкліда для $M = 48$ і $N = 18$ завершується зі значеннями $M = N = 6$. У програмі перевірки парності початкове $N = 5$ переходить у $N = 1$, після чого результатом стає $R = 1$.

Приклади команд:

```
sipl run examples/power.sipl --state X=2 --state Y=5
sipl run examples/gcd.sipl --state M=48 --state N=18
sipl trace examples/parity.sipl --state N=5 --json
sipl semantic examples/parity.sipl
pytest --cov=sipl --cov-report=term-missing
```

10. Відтворюваність та обмеження

Залежності й метадані зафіксовано в `pyproject.toml`. Проєкт можна встановити командою `pip install -e ".[dev]"`, після чого доступна команда `sipl`. README містить команди запуску, тестування та опис кодів завершення.

Поточна версія не реалізує статичну типізацію, IDE, debugger і символічне доведення коректності. DOT генерується без вбудованого Graphviz-рендерера. Ці обмеження явно задокументовано й вони не впливають на заявлені модулі.

11. Висновок

Створено працездатний SIPL Language Toolkit: граматика є єдиним джерелом синтаксису, парсер будує типізоване AST з позиціями, інтерпретатор повертає стан і trace, а візуалізація та семантика використовують AST API.

Реалізацію перевірено 44 тестами з покриттям 84%. Підготовлено приклади для захисту й відтворюване встановлення. Виконано завдання на максимальні 10 балів за умовами проєкту.

Використані матеріали

1. М. С. Нікітченко. *Теорія програмування*. 2020. 2. О. Шишацька, А. Криволап, А. Шишацький. *Теорія програмування. Практикум*. 2024. 3. Технічне завдання *SIPL Language Toolkit*, 2026. 4. Документація бібліотеки Lark Parser.